Compiler Project

USER DOCUMENTATION

Version V1.0.0

Released Date 22/01/2023

Prepared by OhNoCamel Team

GOMEZ ROSALES Laura Sofia

LUGO NOMESQUE José Francisco

SHIRAKAWA Jin

MUTHAIYAH Shaiva

NASSIR SHAFI Ayda

BALERAS Timothé

Abstract

The user document describes the design and practical usage of the MinCaml compiler that converts high level source programs into ARM assembly code. The compiler computes essential tasks and complete compilation workflow starting from lexical analysis and parsing to static type checking and intermediate program transformation. All important stages such as lexing parsing k-normalization closure conversion ASML generation is implemented to transform source code into a form suitable for low level execution. The compiler supports essential datatypes and features such as arithmetic operations, if else control structure, functions, arrays and tuples. The user documentation provides clear guidance for installation usage and understanding the capabilities and limitations of the compiler.

Table of content

Abstract 2
INTRODUCTION 5
Purpose of the compiler 5
Scope of the project 5
SYSTEM REQUIREMENT 5
Hardware Requirement 5
Software Requirement 5
INSTALLATION GUIDLINE 6
Environment Setups 6
LANGUAGE SPECIFICATION 6
Tokens and Keywords 6
Data Types 7
Operators 7
Control Structures 7
Functions 7
USING THE COMPILER 8
Compilation steps 8
IMPORTANT COMMANDS 9
Running the backend 9
ERROR HANDLING 10
Error Messages 10
Lexical Errors Messages 10
Syntax Errors Messages 10
Semantic Error Messages 11
Invalid Type Checking 11
Strict Type error: 11
Branches with different type error: 11
Statement condition is not bool error: 11
Argument mismatch error: 12
Arity mismatch error: 12
Invalid insertion type to array error: 12
Accessing a non-array error: 12
Array Index Type Error (for INT) : 12
Tuple size mismatch: 12
Invalid type inside tuple: 12
Warning Messages 12
Floating point parsing 13

INPUT PROGRAM FORMAT **13**
Naming Conventions 13
LiMITATION **13**
Known issues 13
Unsupported Features 13
FUTURE ENHANCEMENT **13**
Planned Features 13
Possible improvements 14
APPENDIX **14**
Reserved Keywords 14
Token LIst 14
Table 1.1 token list **14**
GLOSSARY **14**
Technical terms and definitions 14

INTRODUCTION

Purpose of the compiler

This compiler design project is designed as a first semester programming language and compiler design project for UGAs masters computer science students. The compiler translates programs written in the selected language MinCaml to ARM assembly code to demonstrate how the compiler works from source code to executable- level output, using well defined intermediate representation. In this process the project demonstrates the key stages in compilation such as lexical analysis, parsing, intermediate representation and code generation.

Scope of the project

The scope of this project is limited to the essential components of the compiler as specified in the given documentation, therefore instead of implementing a fully featured compiler the project focuses on tasks including front end analysis , intermediate transformations, and backend code generation. The floating point arithmetic and other advanced features are thus excluded.

SYSTEM REQUIREMENT

Hardware Requirement

The compiler can be built and run in any machine using a UNIX-based operating system or in a virtual machine running UNIX. A minimum of 4GB is recommended for the compilation. ARM architecture is required to natively execute the generated assembly output, otherwise an emulator must be used.

Software Requirement

The system must have a linux environment with the OCaml 5.2.0 compiler installed .An ARM assembly and linker or an ARM emulator can be used to run the generated code. GitLab is used for version control and collaborative development.

INSTALLATION GUIDLINE

Environment Setups

To begin, the compiler requires a working OCaml development environment. OCaml must be installed along with the necessary libraries and build tools. These tools are used for lexical analysis, parsing and building the compiler. Once we installed the necessary tools and libraries the project repository should be cloned from GitLab using the following link, git@gricad-gitlab.univ-grenoble-alpes.fr:lugonomj/oh-no_camel.git.

After cloning from GitLab using the link navigate to the project root directory ocaml , which contains the source file and Makefile. Then using the make command we can build the compiler in the terminal. This command compiles all components of the compiler and generates the executable required to process MinCaml programs.

LANGUAGE SPECIFICATION

Tokens and Keywords

The compiler recognizes identifiers, integer literals, arithmetic operators and punctuation symbols. Identifiers are used to name variables and functions, while integer literals represent the whole number values used in expressions. Arithmetic operators such as addition, subtraction , multiplication and division allow the construction of mathematical expressions. Punctuation symbols like parentheses, commas, and semicolons are used to structure expressions and statements.

In addition to these general tokens the language includes several reserved keywords such as if, then, else, let, in and Array.create are treated specially and can not be used as identifiers. These keywords have predefined meaning in the language and are used to control the program.

These tokens and language grammar form the basic building block of the language syntax and language grammar which allows the compiler to transform the source code in the correct manner.

Data Types

The compiler supports a limited but important set of data types that are sufficient to express the fundamental programming construct in MinCaml. In the following section we discuss the data types which are supported in the compiler.

- ***Integer(Int)*** - type is used to represent whole numbers and primary numeric types supported by the compiler.
- ***Boolean(Bool)*** - type represents logical values, namely true and false used mainly for conditional expressions.
- ***Unit*** - type is used for expressions that do not return a meaningful value, such as certain function calls or sequencing operations.
- ***Function types*** - represent both user defined and predefined external functions.
- ***Tuple*** - enables grouping multiple values into a single structure.
- ***Arrays*** - which are restricted to storing integer values only.

Operators

The compiler supports the basic arithmetic operators, Addition(+), Subtraction(-), Multiplication(*), and Division (/). All arithmetic operations in the compiler are limited to integers only. As a result expressions in MinCaml are evaluated using integer arithmetic.

Float was given as an option in the given task list therefore implementation of float numbers results in compilation error or unexpected results .

Control Structures

Control flow in MinCaml is primarily handled through conditional expression using if-then -else constructs. The condition expression must be evaluated to boolean values. These structures allow branching logic within MinCaml programs,which depending on the result of the condition, either the then or else branch will be executed. Thus these control structures enable branching logic.

Functions

The compiler supports both user defined and some predefined external functions. Recursive functions are defined using let rec, which enables the functions to call themselves in repetition and iteration. Multiple arguments and External functions for array creation, input output operations, and mathematical computations such as

create_array,create_float_array,print_int, print_newline,sin, cos,sqr,abs_float, int_of_float, float_of_int,truncate are also supported through a predefined interfaces. The external functions list can be extended for further use of other external functions on need.

USING THE COMPILER

Compilation steps

To compile a MinCaml program, first we ensure that the compiler has been built successfully , and this can be done using a make command in the terminal from the project directory ocaml. Once the executable is available it can be called by ./ for this project the executable is mincamlc therefore it will be ./mincamic ../mincaml/filename.ml where filename.ml refers to MinCaml source file to be compiled.

After invocation, the compiler then compiles the input program through a well defined compilation stage as follows.

- ***Lexing***- where the source code is scanned and converted into a sequence of tokens.
- ***Parsing*** - uses the input from the lexing output which analyzes the tokens stream and constructs a syntactic structure based on the language grammar.
- ***Type checking*** - performed to ensure all expressions and operations are type correct according to the static type system.
- ***K-normalization*** - this step performs the simplification of complex expressions by breaking to smaller explicit steps.
- ***Alpha Conversion*** - to rename variables and avoid naming conflicts for the later transformations.
- ***Nested let reduction -*** which simplifies nested let expressions to improve the structure of intermediate representations.
- ***Closure conversion*** - this stage is performed to transform functions into closures, making free variables explicit and providing output for low level code generation.
- ***ASML generation*** - the output of the closure then converted in to ASML which closely represent machine level operations
- ***Optimisation local -*** the compiler applies a local optimisation of the ASML code.

  That can be bypassed.

- ***ARM assembly generation -*** the compiler then generates ARM assembly code which is the low level output that can be executed directly on ARM architecture.

IMPORTANT COMMANDS

Running the backend

1. **mincamlc**

- **Purpose**: This command runs the backend by providing an input ASML file to convert to ARM. The frontend is not processed with this sequence of commands.
- **Syntax:** While in ocaml directory, type this command.

  ./mincamlc [flags] [arguments] [input_file name] -o [output_file name]

- **Sample Usage:** This takes an input ASML file (rec4.asml) from the source and sends it to a folder called ARM under the name output.s. User must be in the ocaml directory and have typed 'make'.

  ./mincamlc -asml_input_mode -reg_alloc lc ../tests/asml_backend/valid/rec4.asml -o ../ARM/output.s

- **Flags:**

  -asml_input_mode: To signal to the compiler that we want the backend to process ASML files directly.

  -reg_alloc [params: lc/spill]: Enables switching between total spilling method and greedy linear allocation.

2. **qemu-arm**

- **Purpose**: This is an emulator which can be used to run an ARM file in ubuntu linux.
- **Syntax:** While in the ARM directory with the ARM file, type make and then type the command in the format provided.

  qemu-arm ./[file_name].arm

- **Sample Usage:** This executes the file named 'output.arm' using an emulator.

  qemu-arm ./output.arm

ERROR HANDLING

Error handling is a process by which the compiler detects and reports and responds to problems regarding source code. In the following section we then discuss the different type messages generated by the compiler to inform the user what went wrong and where.

Error Messages

Error Message is information to let users know about an error occurrence when the compiler detects something that prevents it from continuing compilation.

Lexical Errors Messages

Lexical errors occur due to invalid characters or malformed integer literals .These types of errors are detected during the initial scanning phase in lexer.

Handling float(since the compiler doesn't support float operations)

- Attempting to run float number source codes will be considered invalid but instead of crushing the compiler the following message will be presented.
- *Error Message: "Floating numbers are not supported: might result error”*

Invalid typing and symbols

- The lexer will issue error message when encountered with unknown tokens , unsupported language features , invalid combination of characters or invalid or unsupported characters which should not be their;
- *Error Message: "unknown token " near characters 40-41"*

Syntax Errors Messages

Syntax errors are raised when the input program violates grammatical rules, such as missing keywords and incorrect expressions.

Invalid Tokens

- When the compiler matches the grammar rule but is encountered with symbols which are produced by the lexers but cannot be interpreted by the parser according to the defined language syntax, the compiler will raise the following error message.
- *Error Message: "parse error near characters 72-76”*

Invalid function definitions

The function definition error will be raised when the input does not match the syntax of a valid function definition.

- *Error Message: "parse error for function definition near characters 16-17"*

Invalid function arguments

Function argument error will be raised during parsing when the parser is reading the argument list but doesn't follow the expected grammar.

- *Error Message:* "parse error the input argument doesn't follow the grammar rule near characters 13-14"

Semantic Error Messages

In the semantic error the compiler handles meaning rather than form or grammar. This includes using the wrong types in operations, performing invalid actions or calling functions incorrectly. Most of these errors are also caught during type checking.

For detailed typechecking tests and errors please refer to compiler project developers documentation.

Invalid Type Checking

Note for the following error messages the variables x and y are used as different data type notations (eg int and bool)

Strict Type error:

Type of error raised when the input type doesn't match the type expected by the operation.

- *Error Message: "[Type Error] Mismatch between: x and: y"*

Branches with different type error:

Type of error where two branches the if and the else contain different datatypes for the same variable causing a type mismatch.

- *Error Message: "[Type Error] Mismatch between: x and: y"*

Statement condition is not bool error:

Type of error where non-boolean expression is used instead of boolean condition, causing a type mismatch.

- *“Error Message: [Type Error] Mismatch between: int and: bool”*

Argument mismatch error:

Type of error caused when the type of the variables doesn't match with the expected datatype in the given argument.

- *“Error Message: [Type Error] Mismatch between: x and: y”*

Arity mismatch error:

Type error caused when a function is called with different numbers and types of parameters are mismatched.

- *“Error Message: [Type Error] Mismatch between: (? -> ?) and: (int -> int -> ?)”*

Invalid insertion type to array error:

As the name suggests the error is caused when we try to insert invalid values which have different data types then the array's declared element, causing a type mismatch.

- *“Error Message: [Type Error] Mismatch between: x and: y”*

Accessing a non-array error:

Type of error caused when we try to access an array element. This type of error occurs when an indexing operation is applied to a value that is not an array causing a type mismatch.

- *“Error Message: Type Error] Mismatch between: intr and: [?]”*

Array Index Type Error (for INT) :

Type of error occurs when an array is indexed using a value that is not type integer causing a data mismatch.

- *“Error Message: [Type Error] Mismatch between: bool and: int”*

Tuple size mismatch:

The type of error raised due to mismatch in the number of elements provided does not match the size of the tuple declared.

- *“Error Message: [Type Error] Mismatch between: (? * ? * ?) and: (int * int)”*

Invalid type inside tuple:

Type of error raised due to mismatch in an element in tuple with the tuple’s declared element types.

- *“Error Message: Type Error] Mismatch between: bool and: int”*

Warning Messages

A warning message is a type of message generated by the compiler to inform the user about unexpected behaviour but when the compiler wont be blocked or will continue.

Floating point parsing

Since the floating operation is not handled by the compiler such issue will raise error in lexical analysis stage. But if in some cases such as partial parsing if the lexer produces a token from floating point operation the parser will then see it and will issue a warning instead of error, which then be skipped and continued with other operations.

- *Warning Message warning: float operation is not supported, skipping@*

INPUT PROGRAM FORMAT

Naming Conventions

The following are the list of rules defined regarding the naming conventions.

- Reserved names cannot be used as identifiers
- Variable and function names are case sensitive
- Identifier names must start with letters
- Any floating number usage result an error

LiMITATION

Known issues

- Closure in the back end
- Strict Syntax dependency
- Memory-intensive programs cannot be executed with total-spill allocation

Unsupported Features

- Floating-point operation

FUTURE ENHANCEMENT

Planned Features

In the future version floating- point support, improved error diagnostic, and additional optimization can be included.

Possible improvements

Potential improvements include advanced register allocation strategies and extending the standard library to be able to make the compiler fully functional compiler are some of the possible improvement points.

APPENDIX

Reserved Keywords

IF(if), THEN (then), ELSE (else), LET (let), REC (rec)

Token LIst

Literals	Arithmetic Operators	Logical Operators
INT FLOAT BOOL	PLUS(+) MINUS(-) AST(*) SLASH(/) PLUS_DOT (+.) AST_DOT(*.) SLASH_DOT(/.)	NOT(not)
Comparison Operators	Identifiers	Array and Assignment
EQUALS (=) LESS (<) GREATER (>) LESS_EQUAL (>=) GREATER_EQUAL (<=)	IDENT(string) IDENT(Id.genid()) - wild card identifiers	ARRAY_CREATE (Array.create) LESS_MINUS(<-)

Table 1.1 token list

GLOSSARY

Technical terms and definitions

- **AST** - Abstract Syntax Tree, a structure representation of source code.
- **ASML** - An intermediate assembly language used between front end and back end.